

Gestion de code avec Git (FIEC30EM)

frank.buloup@univ-amu.fr

Aix Marseille Université
Faculté des Sciences du Sport
Master 2 IEAP - Facteurs Humains



Gestion de code avec Git

- 1 Introduction
- 2 Utilisation & Installation de Git
- 3 Annexes

1

- Organisation & Objectifs du module
- Le compte GitHub & les projets
- Git ? C'est quoi ?
- Petit historique des systèmes de contrôle de versions

À l'issue de cette formation, vous serez capable de :

- 1 Décrire les principes de fonctionnement d'un gestionnaire de versions distribué
- 2 Créer, initialiser et cloner un dépôt avec Git
- 3 Manipuler les commandes de Git pour gérer les fichiers et les branches
- 4 Résoudre les conflits de fusion
- 5 Mettre en œuvre un projet en mode collaboratif avec Git sur GitHub

Compte GitHub

- Login : MasterFHGit
- Password : MasterFHGit2025

Les projets

Sur la base d'un Arduino Uno R4 Wifi avec :

- une jauge de contrainte (2 postes)
- un potentiomètre
- une résistance à détection de force (2 postes)
- un capteur à ultrason (2 postes)
- un codeur optique



git

Une idée ?

Documentation Git

Git n'est pas un serveur

- Git n'offre pas de fonctionnalité de gestion de projet complète (e.g. gestion de tâches, de bugs ou wiki)
- Github et GitLab sont deux exemples de solutions complètes



Github



GitLab

Git n'est pas un serveur

- Git n'offre pas de fonctionnalité de gestion de projet complète (e.g. gestion de tâches, de bugs ou wiki)
- Github et GitLab sont deux exemples de solutions complètes



Github



GitLab

Git n'est pas un espace de stockage de fichiers type Dropbox

- Pas de sauvegarde automatique des fichiers
- Pas de gestion optimale des fichiers volumineux* ou binaires**

* Utilisation d'une extension spécifique

**** Les fichiers binaires ne sont pas très lisibles (git diff !!!)**

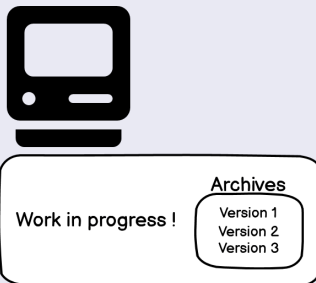
Git est un système de contrôle de version (Version Control System)

- Historique des versions de tous les fichiers
- Manipulation complète de l'historique possible
- Décentralisé (bientôt plus clair...)
- On trouve aussi : "Source Code Management System"

Un peu d'histoire !

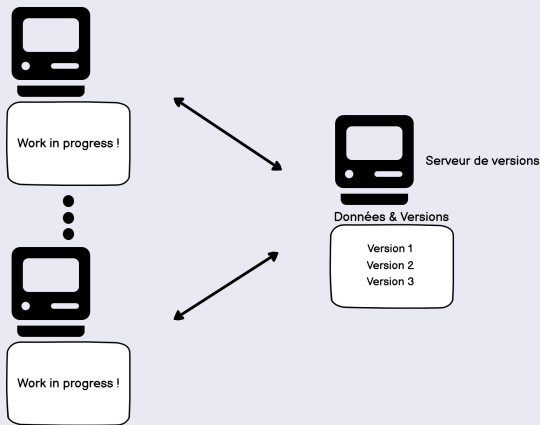
Local VCS - Avant... il y a longtemps !

- Système D : horodatage, classement dossiers, compression etc.
- Pro : Revision Control System (1982, projet GNU*)



* GNU : GNU's Not Unix

- Partage plus facile
- Connexion réseau obligatoire (approche Client-Serveur)



e.g. Concurrent Version System (CVS), Subversion (SVN)

L'origine de Git

- Créé en 2005 par [Linus TORVALDS](#)
- Pour le noyau Linux
- Activement maintenu par [Junio HAMANO](#)
- Popularisé par GitHub
- On prononce "guit" et pas "jit" !
- Git = Crétin (= Conna... !)

Git - Wikipedia

Les principales caractéristiques de Git

- Contrôle de version distribué (pair-à-pair)
- Gestion des branches de code (création, suppression, fusion)
- Opérations atomiques (Commits)
- Données de gestion stockée dans un répertoire .git

2 Utilisation & Installation de Git

- Les différents modes d'utilisation
- Utilisation sur GitHub
- Installation & utilisation en local - Les commandes de base
- Les zones de Git
- Annuler un commit : commande revert
- Modifier l'historique des commits : commande reset
- Fusionner et rebaser des branches - Résolution de conflits
- Collaboration

Comment utiliser Git ?

- En ligne de commande (Terminal & Git Bash pour Windows)
- Outils graphiques intégrés à Git : "git-gui" et "gitk"
- Avec une IHM dans un OS (GitKraken, SourceTree) : [ici](#)
- Intégré dans un EDI (plugin, package etc.)
- Sur internet : github, gitlab, bitbucket etc.



GitKraken



SourceTree

L'apprentissage en ligne de commande est le plus efficace !

Exercice I - Premier pas avec GitHub

- 1 Créer un compte sur GitHub
- 2 Créer un premier dépôt public dans GitHub nommé "FirstRepository"

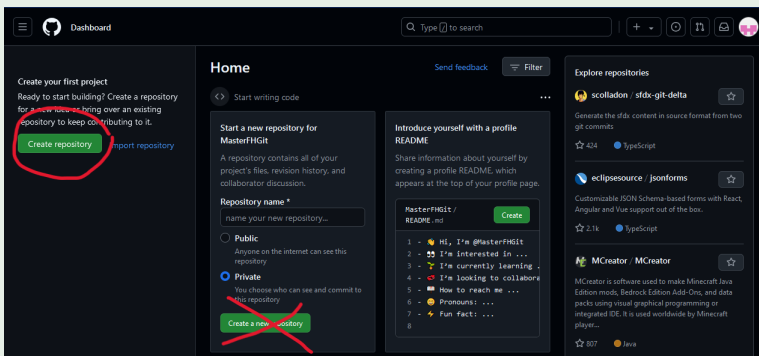


Figure: Création d'un dépôt dans Github

Required fields are marked with an asterisk (*).

Owner *

MasterFHGit -

Repository name *

FirstRepository

FirstRepository is available.

Great repository names are short and memorable. Need inspiration? How about [improved-octo-winner](#) ?

Description (optional)

☒ Public

Anyone on the internet can see this repository. You choose who can commit.

☐ Private

You choose who can see and commit to this repository.

Initialize this repository with:

☒ Add a README file

This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template: None

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

License: None

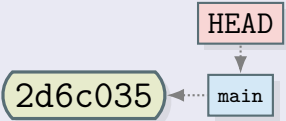
A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

This will set `main` as the default branch. Change the default name in your [settings](#).

You are creating a public repository in your personal account.

Create repository

Figure: Création du dépôt "FirstRepository" public avec fichier README



- **2d6c035** est l'ID du commit (l'ID complet est un SHA1)
- **main** est une référence au dernier commit de la branche qui porte le même nom, sur le dépôt distant
- On peut avoir plusieurs branches et donc plusieurs références (**main**, **dev**, **feature1** etc.)
- On trouvera aussi **master** à la place de **main**
- **main** ou **master** sont par convention des noms de branches de production

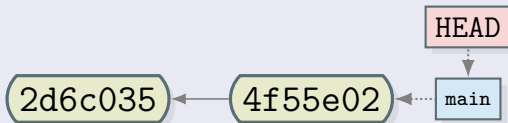
Exercice I - Premier pas avec GitHub

- ### 3 Modifier le fichier README.md sur GitHub

Exercice I - Premier pas avec GitHub

- 3 Modifier le fichier README.md sur GitHub

État après le deuxième commit sur github



Heads

A head is simply a reference to a commit object. Each head has a name. By default, there is a head in every repository called master (or main). A repository can contain any number of heads. At any given time, one head is selected as the “current head.” This head is aliased to HEAD, always in capitals.

Note this difference: a “head” (lowercase) refers to any one of the named heads in the repository; “HEAD” (uppercase) refers exclusively to the currently active head. This distinction is used frequently in Git documentation.

Git - Heads

Exercice II - Installation de Git en local

1 Installer Git en suivant le lien suivant : [Git - SCM](#)

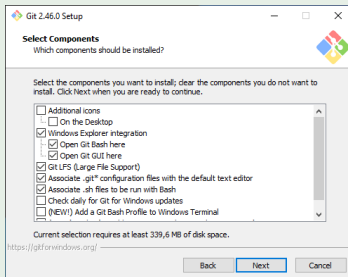


Figure: Git Setup - Garder les options par défaut

- 2 Créer un dossier GIT sur votre machine
- 3 Importer votre dépôt FirstRepository dans ce dossier

Figure: Commandes clone et remote

27 / 83

remote "origin" ?

- origin est un alias local donné au dépôt distant
- il peut être changé dans le fichier config ...

```
MINGW64:/c:/Users/Buloup/Documents/GIT/FirstRepository/.git
Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository/.git (GIT_DIR!)
$ vim config
Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository/.git (GIT_DIR!)
$ cat config
[core]
    repositoryformatversion = 0
    filemode = false
    bare = false
    logallrefupdates = true
    symlinks = false
    ignorecase = true
[remote "toto"]
    url = https://github.com/MasterFHGit/FirstRepository.git
    fetch = +refs/heads/*:refs/remotes/origin/*
[branch "main"]
    remote = toto
    merge = refs/heads/main
Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository/.git (GIT_DIR!)
$ |
```

Figure: Changer l'alias origin

... ou lors de la commande clone

```
MINGW64:/c/Users/Buloup/Documents/GIT/FirstRepository
Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT
$ git clone https://github.com/MasterFHGit/FirstRepository.git -o toto
Cloning into 'FirstRepository'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT
$ cd FirstRepository/

Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository (main)
$ git remote -v
toto    https://github.com/MasterFHGit/FirstRepository.git (fetch)
toto    https://github.com/MasterFHGit/FirstRepository.git (push)

Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository (main)
$ cat ./.git/config
[core]
    repositoryformatversion = 0
    filemode = false
    bare = false
    logallrefupdates = true
    symlinks = false
    ignorecase = true

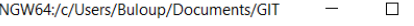
[remote "toto"]
    url = https://github.com/MasterFHGit/FirstRepository.git
    fetch = +refs/heads/*:refs/remotes/toto/*

[branch "main"]
    remote = toto
    merge = refs/heads/main

Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT/FirstRepository (main)
$ |
```

Figure: Changer l'alias origin

Comment supprimer un dépôt local ?



```
MINGW64: c:/Users/Buloup/Documents/GIT

Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT
$ rm -R -f FirstRepository/

Buloup@Prec7670-Buloup MINGW64 ~/Documents/GIT
$ |
```

Figure: Supprimer un dépôt local

branch, main, fetch, push, merge etc. ?

C'est le vocabulaire des DVCS (Git)
Plus clair progressivement !

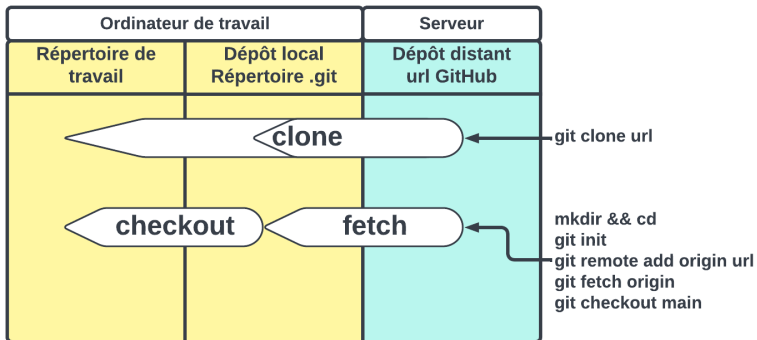


Figure: Importer un dépôt distant - Les deux versions

- 1 Changer le fichier README sur github (branche main)
- 2 Reporter ces changements en local

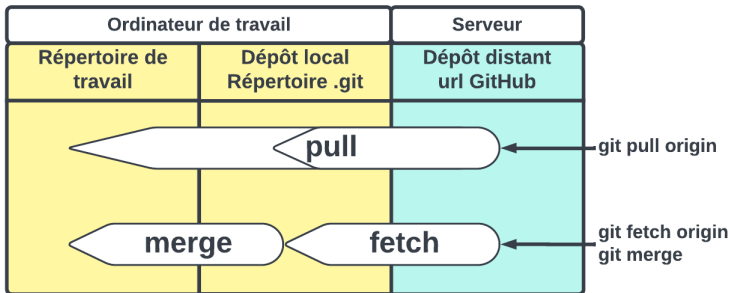


Figure: Mettre à jour un dépôt local - Les deux versions

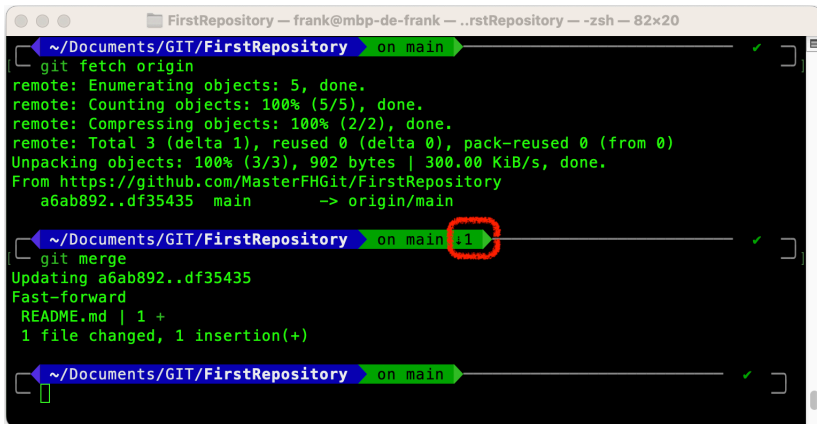
```

~/Documents/GIT/FirstRepository on main ✓
git pull origin
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 904 bytes | 301.00 KiB/s, done.
From https://github.com/MasterFHiG/FirstRepository
   4c8e792..a6ab892  main       -> origin/main
Updating 4c8e792..a6ab892
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)

~/Documents/GIT/FirstRepository on main ✓

```

Figure: Mettre à jour un dépôt local - Version 1

A terminal window titled "FirstRepository — frank@mbp-de-frank — ..rstRepository — -zsh — 82x20". The terminal shows the execution of two Git commands. The first command is "git fetch origin", which outputs: "remote: Enumerating objects: 5, done.", "remote: Counting objects: 100% (5/5), done.", "remote: Compressing objects: 100% (2/2), done.", "remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)", "Unpacking objects: 100% (3/3), 902 bytes | 300.00 KiB/s, done.", and "From https://github.com/MasterFHGit/FirstRepository a6ab892..df35435 main -> origin/main". The second command is "git merge", which outputs: "Updating a6ab892..df35435", "Fast-forward", "README.md | 1 +", and "1 file changed, 1 insertion(+)". The terminal has a dark background with green text. The prompt is "~ / Documents / GIT / FirstRepository" and the branch is "on main". The terminal window has a title bar with standard macOS window controls.

```
FirstRepository — frank@mbp-de-frank — ..rstRepository — -zsh — 82x20

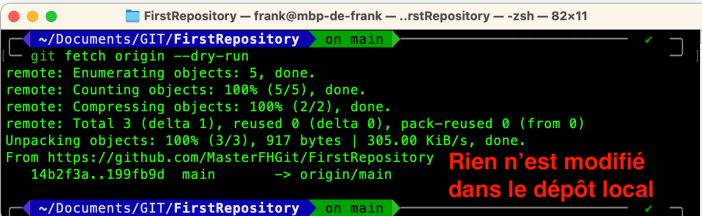
~/Documents/GIT/FirstRepository on main ✓
└─ git fetch origin
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 902 bytes | 300.00 KiB/s, done.
From https://github.com/MasterFHGit/FirstRepository
a6ab892..df35435 main -> origin/main

~/Documents/GIT/FirstRepository on main ✓
└─ git merge
Updating a6ab892..df35435
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)

~/Documents/GIT/FirstRepository on main ✓
└─
```

Figure: Mettre à jour un dépôt local - Version 2

Commande fetch & option --dry-run



```
FirstRepository — frank@mbp-de-frank — ..rstRepository — -zsh — 82x11

~/Documents/GIT/FirstRepository on main
└─ git fetch origin --dry-run
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 917 bytes | 305.00 KiB/s, done.
From https://github.com/MasterFhGit/FirstRepository
  14b2f3a..199fb9d  main       -> origin/main

~/Documents/GIT/FirstRepository on main
```

Rien n'est modifié dans le dépôt local

Figure: Le dépôt local est-il à jour ? Non !



```
FirstRepository — frank@mbp-de-frank — ..rstRepository — -zsh — 82x13

~/Documents/GIT/FirstRepository on main
└─ git pull origin
From https://github.com/MasterFhGit/FirstRepository
  14b2f3a..199fb9d  main       -> origin/main
Updating 14b2f3a..199fb9d
Fast-forward
 README.md | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)

~/Documents/GIT/FirstRepository on main
└─ git fetch origin --dry-run

~/Documents/GIT/FirstRepository on main
```

Figure: Maintenant oui !

Petit résumé intermédiaireire

Quelques commandes : pull, fetch

```
1 $ git pull origin           # Get from origin
2 $ git fetch origin && git merge # Idem !
3 $ git fetch origin --dry-run  # Has origin changed ?
4 $ git log                    # Print commits
5 $ git log --oneline          # Print abbrev-commits
```

Exercice IV - Changements à partir du dépôt local

- 1 Changer le fichier README en local (branche main)
- 2 Reporter ces changements en distant

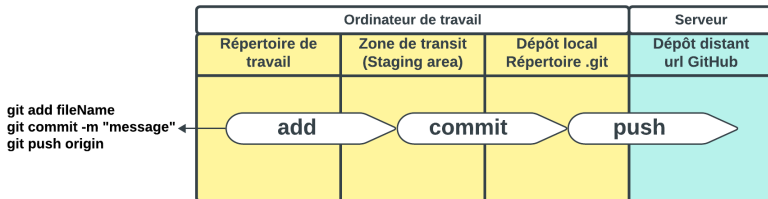


Figure: Mettre à jour le dépôt distant

```

FirstRepository — frank@mbp-de-frank — ..rstRepository — -zsh — 98x33

~/Documents/GIT/FirstRepository on main ✓ took 11s
clear

~/Documents/GIT/FirstRepository on main ✓
~/micro README.md

~/Documents/GIT/FirstRepository on main !1 ✓ took 7s
git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")

~/Documents/GIT/FirstRepository on main !1 ✓
git add README.md

~/Documents/GIT/FirstRepository on main +1 ✓
git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   README.md

~/Documents/GIT/FirstRepository on main +1 ✓

```

Figure: Zone de transit

[illegible]

```

FirstRepository — frank@mbp-de-frank — ..rstRepository — -zsh — 98x37

~/Documents/GIT/FirstRepository on main ✓
~/micro README.md

~/Documents/GIT/FirstRepository on main !1 ✓ took 8s
git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")

~/Documents/GIT/FirstRepository on main !1 ✓
git add README.md

~/Documents/GIT/FirstRepository on main +1 ✓
git commit README.md -m "Changed from local repository"
(main 47b514b) Changed from local repository
1 file changed, 1 insertion(+)

~/Documents/GIT/FirstRepository on main !1 ✓
git push origin
Username for 'https://github.com': MasterFHGit
Password for 'https://MasterFHGit@github.com':
remote: Support for password authentication was removed on August 13, 2021
remote: Please see https://docs.github.com/get-started/getting-started-with-git/about-remote-repos
itories#cloning-with-https-urls for information on currently recommended modes of authentication.
fatal: Authentication failed for 'https://github.com/MasterFHGit/FirstRepository.git/'

~/Documents/GIT/FirstRepository on main +1 128 x took 24s

```

Figure: Mettre à jour le dépôt distant

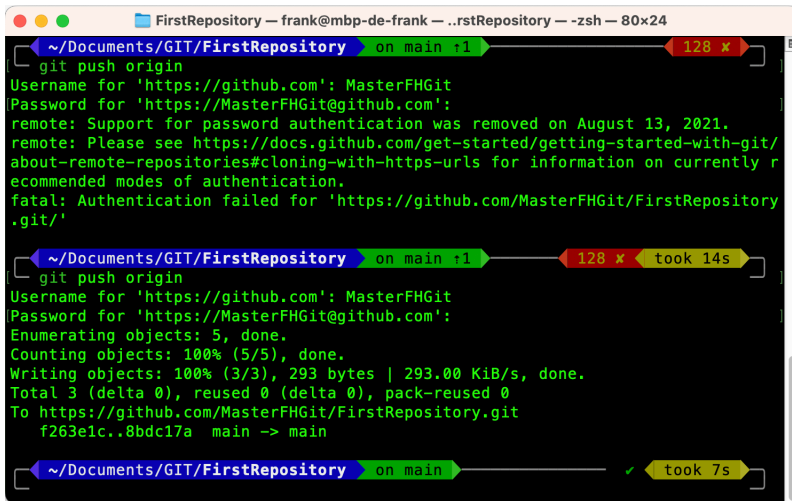


Figure: Génération d'un jeton dans GitHub

Génération d'un jeton d'accès personnel

- Conserver ce jeton. Il n'est visible qu'une seule fois !
- C'est bien de lui donner un petit nom (note) !
- Ne pas oublier de sélectionner l'option "repo"
- Sur un dépôt privé, rien n'est possible sans jeton

Pour plus d'infos : [Personal Access Token \(PAT\)](#)



The image shows a terminal window titled "FirstRepository — frank@mbp-de-frank — ..rstRepository — -zsh — 80x24". It displays two attempts to run `git push origin`. The first attempt fails with a "fatal: Authentication failed" error. The second attempt succeeds, showing progress bars for enumerating, counting, and writing objects, and a final commit message. The terminal output is as follows:

```
~/Documents/GIT/FirstRepository on main +1 128 x
git push origin
Username for 'https://github.com': MasterFHGit
Password for 'https://MasterFHGit@github.com':
remote: Support for password authentication was removed on August 13, 2021.
remote: Please see https://docs.github.com/get-started/getting-started-with-git/about-remote-repositories#cloning-with-https-urls for information on currently recommended modes of authentication.
fatal: Authentication failed for 'https://github.com/MasterFHGit/FirstRepository.git/'

~/Documents/GIT/FirstRepository on main +1 128 x took 14s
git push origin
Username for 'https://github.com': MasterFHGit
Password for 'https://MasterFHGit@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (3/3), 293 bytes | 293.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/MasterFHGit/FirstRepository.git
   f263e1c..8bdc17a  main -> main

~/Documents/GIT/FirstRepository on main ✓ took 7s
```

Figure: Avec le jeton comme mdp, le push fonctionne

Pour enregistrer le jeton

```
1 $ git config --global credential.helper store
```

... et ne plus avoir à le retaper qu'une seule fois. Le jeton est enregistré dans le fichier texte **.git-credentials** de votre répertoire personnel.

Pour signer les commits automatiquement

```
1 $ git config --global user.email "mail@somewhere.com"  
2 $ git config --global user.name "username"
```

Signed-off-by: username <mail@somewhere.com>

Pour cloner (ou autre) on peut aussi faire :

```
1 $ git clone https://uname:pass@github.com/uname/rp.git
```


Petit résumé intermédiaireire

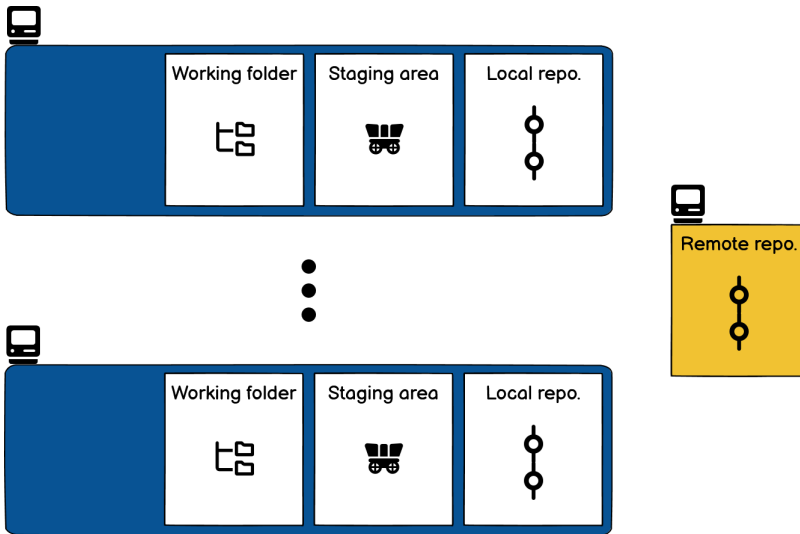
Quelques commandes : add, commit

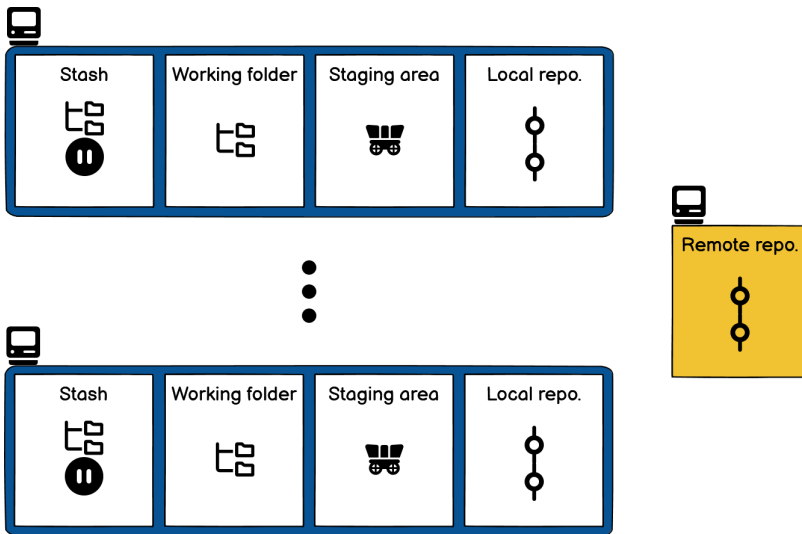
```
1 $ git add fileName # Stage fileName
2 $ git add .         # Stage modified and new files
3 $ git add -u        # Stage modified and deleted files
4 $ git add -A        # Stage everything
5 $ git commit -m "A commit message"
```

D'autres commandes utiles ! switch, branch, checkout

```
1 $ git switch branchName      # Switch to branchName
2 $ git checkout 52fd6e4       # In detached HEAD
3 $ git checkout branchName    # HEAD on branchName
4 $ git branch -D branchName   # To delete branchName
5 $ git branch                 # List all local branches
6 $ git branch -r              # List all remote branches
7 $ git branch -a              # List all branches
8 $ git branch newBranchName   # Create new branch
9 $ git switch -c neBrNa       # Create & switch to neBrNa
```

Les zones de Git





stash = réserve, remise

Exercice V - Commande revert

- ➊ Ajouter un nouveau commit dans la branche **main** en modifiant le fichier README.md (Before revert)
- ➋ Utiliser la commande revert pour annuler ce commit
- ➌ Utiliser la commande checkout pour vous déplacer dans les commits

Exercice V - Commande revert

- ➊ Ajouter un nouveau commit dans la branche **main** en modifiant le fichier README.md (Before revert)
- ➋ Utiliser la commande revert pour annuler ce commit
- ➌ Utiliser la commande checkout pour vous déplacer dans les commits

Commande revert

```
1 $ git switch main           # Goto main branch
2 $ vim README.md             # Modify README
3 $ git commit -am "Before revert" # Commit
4 $ git revert HEAD           # Revert last commit
5 $ git log --oneline         # See logs
6 $ cat README.md             # File content
7 $ git checkout 44e5761      # Goto previous commit
8 $ cat README.md             # File content
```

Commande revert

- Elle ne modifie pas l'historique des commits (pas de suppression de commits)
- Elle crée un nouveau commit
- Elle détermine comment inverser les changements introduits par le commit spécifié et ajoute un nouveau commit avec le contenu inversé qui en résulte

Exercice VI - Commande reset

- 1 Modifier le fichier README.md sans faire de commit
- 2 Supprimer les deux derniers commits (le Before revert et le revert) précédents en utilisant la commande reset
- 3 Conclusion

Exercice VI - Commande reset

- 1 Modifier le fichier README.md sans faire de commit
- 2 Supprimer les deux derniers commits (le Before revert et le revert) précédents en utilisant la commande reset
- 3 Conclusion

Commande reset

```
1 $ git checkout main # HEAD to main
2 $ vim README.md # Change README.md
3 $ git log --oneline # See logs
4 $ git reset 9d49258 # Reset two commits
5 $ git log --oneline # See logs
6 $ cat README.md # File content
7 $ git restore README.md # Get from index
8 $ cat README.md # File content
```

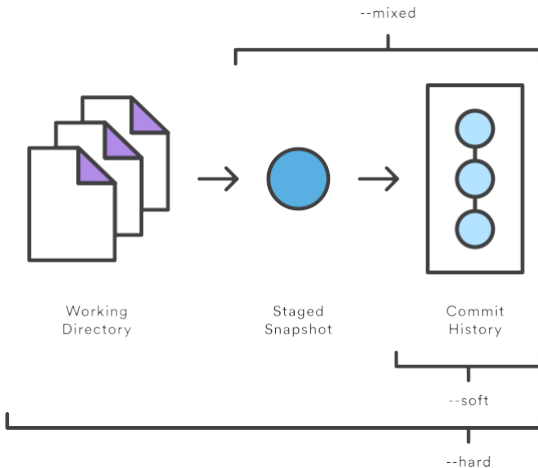


Figure: Commande reset - Les différents modes

Commande reset - Option mixed

- Elle revient dans l'état du commit spécifié en mettant l'index (staging area) à jour mais pas la copie de travail
- Elle modifie l'historique des commits (supprime des commits)
- La commande restore permet de récupérer l'index : `git restore README.md`
- C'est l'option par défaut : mode mixte

"git reset idCommit" et "git reset --mixed idCommit" sont donc des commandes indentiques

Commande reset - Options soft, mixed et hard

- Récupérer l'état de certains fichiers et conserver les modifications en cours pour les autres fichiers
- Réécrire l'historique des commits dans le dépôt (e.g. regrouper des commits)

Comment annuler un reset ?

Quelle que soit l'option utilisée, il est toujours possible d'annuler un reset en retrouvant l'ID des commits par la commande `reflog`

```
1 $ git reflog # get idCommit just before reset
2 $ # reset hard to this commit
3 $ git reset --hard idCommit
```

--hard reste une option dangereuse !

Exercice VII - Préparation fusion et rebase

- ## 1 Ajouter un nouveau commit dans la branche **dev**

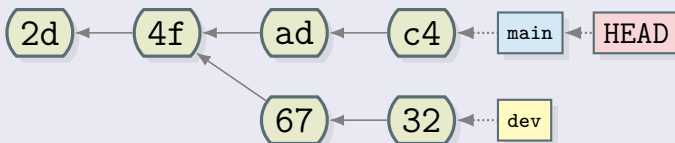
Exercice VII - Préparation fusion et rebase

- ## 1 Ajouter un nouveau commit dans la branche **dev**

Préparation fusion et rebase

```
1 $ git switch dev           # Goto dev branch
2 $ vim README.md           # Modify README
3 $ git commit -am "devC1"  # Commit 1 in dev branch
```

État actuel du dépôt local



Comment récupérer les modifications de **dev** dans **main** ?

- Les branches **dev** et **main** ont divergées
- Problème de fusion de branches : gestion des conflits

Exercice VIII - Fusion de branches avec conflits

- ## 1 Fusionner la branche dev dans la branche main en local

Fusionner : la commande merge

```
1 $ git switch main    # Goto main branch
2 $ git merge dev      # Merge dev branch to main branch
```

Exercice VIII - Fusion de branches avec conflits

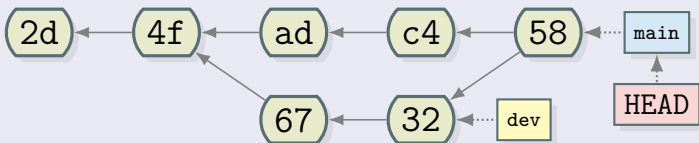
- 1 Fusionner la branche dev dans la branche main en local
- 2 Modifier le fichier README.md
- 3 Ajouter ce fichier au staging (marquer la résolution de conflits)
- 4 Faire le commit

Fusionner : la commande merge

```
1 $ git switch main      # Goto main branch
2 $ git merge dev        # Merge dev branch to main branch
3 $ vim README.md        # Modify README.md
4 $ git add README.md    # To mark resolution
5 $ git commit -m "dev->main,,Conflicts,,resolution"
```

Ne pas hésiter à utiliser la commande `git status` régulièrement

État du dépôt local après la fusion - Merge



Les commits de la branche *dev* ont été fusionnés avec la branche *main* dans le dernier commit de cette branche (58)

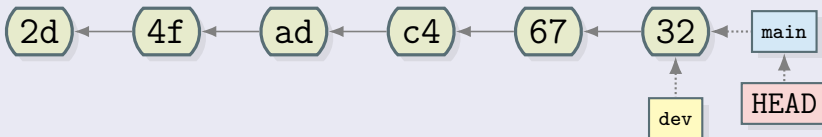
Exercice IX - Rebase de branches avec conflits

- 1 Revenir dans l'état du dépôt distant (supprimer puis cloner de nouveau le dépôt distant)
- 2 Ajouter un nouveau commit dans la branche **dev** en local
- 3 Rebase la branche dev dans la branche main en local
- 4 Modifier le fichier README.md
- 5 Ajouter ce fichier au staging pour marquer la résolution de conflits
- 6 Terminer le rebase

Rebaser : la commande rebase

```
1 $ git switch dev # Goto dev branch
2 $ vim README.md # Edit readme
3 $ git commit -am "devC1" # New commit in dev
4 $ git switch main # Goto main
5 $ git rebase main dev # Rebase dev in main
6 $ vim README.md # Resolve conflicts
7 $ git add README.md # Mark resolved
8 $ git rebase --continue # Continue rebase
9 $ git switch main # Goto main
10 $ git merge dev # Merge dev in main
11 $ git branch -d dev # Delete dev (-D?)
```

État du dépôt local après le rebase



Tous les commits de la branche *dev* ont été mis à la suite de ceux de la branche *main*



Figure: Illustration des commandes merge à gauche et rebase à droite

3 Annexes

- Oh my ¿what? !
- Applications utiles
- Créer un dépôt Github à partir d'un dépôt local
- Lecture de la sortie d'un diff
- Lecture de la sortie d'un pull
- Une longue ligne de commande !
- Les accroches - Hooks
- Les alias
- Cueillons des cerises !
- La remise
- Comment dire à git d'ignorer des fichiers ?
- Quel casse tête tous ces HEAD !
- Vidéos sur Delicious Insights



Figure: Logiciels SourceTree, GitHub et GitKraken Desktop

Trois outils graphiques libres et très utiles

Contexte

J'ai des fichiers sur mon ordi que je souhaite gérer sur GitHub

Solution

- 1 Créer le dépôt local si pas déjà fait
- 2 Créer le dépôt distant sur GitHub **VIDE** (évite les erreurs) :
 - Pas de fichier README.md, .gitignore, licence etc.
 - Récupérer l'URL du dépôt distant
- 3 Ajouter cet URL en tant que dépôt distant
- 4 Pousser la branche principale sur le dépôt distant

```
1 $ git init
2 $ git add ...
3 $ git commit ...
4 $ git remote add origin REMOTE-URL
5 $ git remote -v # Check URL has been added
6 $ git push -u origin main
```

```
1 $ git diff 7e46066..3a78b4a # Changes between commits
```

```

\subsection{Utilisation sur GitHub}
@@ -479,7 +484,7 @@ Note this difference: a "head" (lowercase) refers to any one of the named he
\begin{alertblock}{remote "origin" ?}
\begin{itemize}
\item origin est un alias local donné au dépôt distant
- \item il peut être change dans le fichier config
+ \item il peut être changé dans le fichier config ...
\center{
\includegraphics[scale=0.5]{images/origin1.PNG}
\captionof{figure}{Changer l'alias origin}}
@@ -490,15 +495,14 @@ Note this difference: a "head" (lowercase) refers to any one of the named he
\end{frame}

\begin{frame}
-\begin{alertblock}{remote "origin" ?}
-\begin{itemize}
- \item ou lors de la commande clone
+\begin{alertblock}{... ou lors de la commande clone}
\center{
\includegraphics[scale=0.45]{images/origin2.PNG}
\captionof{figure}{Changer l'alias origin}}
-\end{itemize}
-\centering
-
\end{alertblock}
\end{frame}

@@ -554,7 +554,7 @@ $ git switch branchName # Move HEAD to branchName

```

-479,7 : 7 lignes **à partir de** la 479 dans le commit 7e46066

+484,7 : 7 lignes à partir de la 484 dans le commit 3a78b4a

```
1 # Changes between workspace and stage
2 $ git diff
3 # Changes between workspace + stage and last commit
4 $ git diff HEAD
```

```
diff --git a/presentation.pdf b/presentation.pdf
index eb2bfe9..326de97 100644
Binary files a/presentation.pdf and b/presentation.pdf differ
diff --git a/presentation.tex b/presentation.tex
index a02e9fe..4f1635d 100644
--- a/presentation.tex
+++ b/presentation.tex
@@ -1165,36 +1165,6 @@ Pour plus d'infos : \textcolor{blue}{\textbf{\und
\end{frame}

\section{Les projets}

-
-
-\begin{frame}
-\begin{alertblock}{Rien à disposition pour faire des mesures objectives}
-\begin{itemize}
```

Les index présents en entête de la sortie d'un diff sont des identifiants internes à git, pas des numéros de commits !


```
~/Documents/GIT/CourseMaterial on main ✓ base
git pull
From https://github.com/MasterFHGit/CourseMaterial
 7e46066..3a78b4a  main    -> origin/main
Updating 7e46066..3a78b4a
Fast-forward
 images/GUI.png          | Bin 0 -> 24545 bytes
 images/SaveCentralNew.png | Bin 0 -> 65138 bytes
 images/SaveDistribNew.png | Bin 0 -> 111885 bytes
 images/SaveLocalNew.png  | Bin 0 -> 26221 bytes
 images/devis.png         | Bin 0 -> 75387 bytes
 images/zones1.png        | Bin 0 -> 54719 bytes
 images/zones2.png        | Bin 0 -> 60191 bytes
 presentation.pdf         | Bin 5267356 -> 6625085 bytes
 presentation.tex         | 324 +-----
 presentation.vrb         | 19 +---
10 files changed, 259 insertions(+), 84 deletions(-)
create mode 100644 images/GUI.png
create mode 100644 images/SaveCentralNew.png
create mode 100644 images/SaveDistribNew.png
create mode 100644 images/SaveLocalNew.png
create mode 100644 images/devis.png
create mode 100644 images/zones1.png
create mode 100644 images/zones2.png
```

De nouveaux fichiers ajoutés en mode normal (100644)

343 changements (ajouts et suppressions)

Beaucoup de changements dans presentation.tex

Il existe d'autres modes : e.g. 100755 et 120000

Une longue ligne de commande !

```
* 2186499 Update reset and go further topics
*   af46e63 Merge conflict
| \
| * 7e46066 Add to go further
| * 654b4e9 Change font size and add git SCM link
* | fd02601 Add clone command with uname and passwd
| /
* db8324e Remove pong image
* 7149293 Update revert and reset commands
* e8896ef Update merge and rebase commands
* 48b507e Global update 3
* 6b16542 Global update 2
* e44f652 Global update
* fe4cd53 Ajout des commandes rebase et merge
* b6f584b Add annexes
*   74a5511 Resolve conflict
| \
| * 8971e4d Some minor changes
* | e24286b Update projects
| /
* de0dca4 Remove spaces before $ in lstlisting env.
* dedb9fb Add credential and Signed-Off alerts blocks
* 8ffa17f Update projects part
* 73cc805 Update projects and git commands
* f6ae20c Add main concepts slides
* c979f94 Add gitDAG library to draw nice git graph
```

```
1 $git log --graph --decorate --abbrev-commit --all --pretty=oneline
```

Un superbe graphe de commits dans la console 😊!





Git Alias

```
alias gs=' git status '
alias ga=' git add '
alias ga=' git push'
alias c=' git commit '
```

Documentation Alias sur Git

Les alias en pratique

```
1 # Creating an alias to get a
2 # graph of commits in console
3 $ git config --global alias.lg 'log --graph --decorate
    --abbrev-commit --all --pretty=oneline'
4 $ git lg
```



"Cherry-pick" sur Delicious Insights

Cherry-pick en pratique

Une bonne pratique consiste à créer une branche de report. La Gestion de conflits/fusions est possibles. L'option -x permet d'écrire l'ID du commit initial dans le commit final ("cherry picked from commit commitID").

```
1 $ git switch rBranch # Branch which will receive picks
2 # Create and switch to a report branch from rBranch
3 $ git switch -c rBranchReport
4 $ git cherry-pick -x commitID1 # Pick oldest commit
5 $ git cherry-pick -x commitID2 # Pick next commit
6 $ git cherry-pick -x commitID3 # Pick next commit
7 $ # etc ... Manage possible conflicts etc...
8 $ git switch rBranch # Switch to rBranch
9 $ git merge rBranchReport # Report rBranchReport
10 $ git branch -d rBranchReport # Delete rBranchReport
```



"La remise" sur Delicious Insights

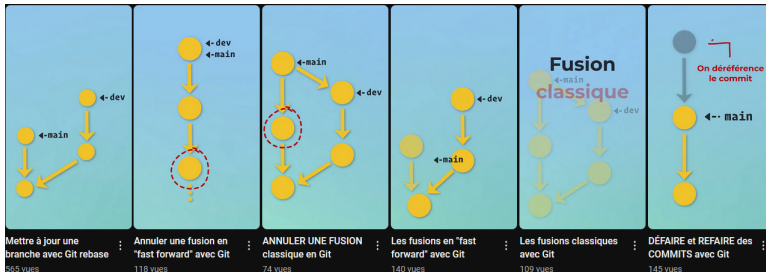


"Ignorer des fichiers" sur Delicious Insights

Quel casse tête tous ces HEAD !

A diagram showing four overlapping circles arranged in a diamond pattern. The top circle is red and labeled **HEAD~2**. The bottom circle is green and labeled **HEAD~1**. The left circle is blue and labeled **HEAD**. The right circle is orange and labeled **HEAD^3**. The circles overlap in the center and at the intersections of their boundaries.

Les différentes formes de HEAD expliquées sur StackOverflow



Shorts Git sur Delicious Insights